DAY 05 · BYTES & UTF-8

Bytes, Bits & UTF-8

Bits, masks, and UTF-8 encoding and decoding by hand.

3 hours / day05_bytes_utf8 / Bytes and UTF-8

INSIDE THIS BOOKLET

Bits · Masks · UTF-8 · Error handling

How to use this booklet

Each day is a working session, not a reading assignment. This booklet is the typeset version of the day's `README.md`: keep the folder open, type the examples into a Python REPL, and run the tests as you go. The *Exam lens* and end-of-day review are study aids added to the original material.

Read & type
Type every example. Predict the output first.

Practice
Implement or debug one test at a time.

Test
Run the suite. Read the first failing line.

Log
Record each bug in `bug_journal.md`.

You will need: Python 3.10+ · a terminal · `day05_bytes_utf8/` · the course runner `python run.py 5`

In this session

The story of today 3

1 Bits, binary and hexadecimal: a gentle start 3

2 `str` and `bytes` in Python 4

3 Bit operations: working with individual bits 7

4 How UTF-8 works 9

5 Other Unicode facts (good to know, rarely tested) 13

6 The toolbox: techniques the exercises rely on 14

Exercises 16

End-of-day checklist 17

> **◎ Today's goal**
>
> The brief explicitly says *"string manipulation including encoding and decoding UTF-8"*. By the end of today you can encode and decode UTF-8 **with pen and paper**, spot a wrong mask or shift in a decoder within seconds, and know every `str`/`bytes` API that matters.

The story of today

Computers don't store letters. They store numbers, and more precisely they store **bits**: tiny switches that are either 0 or 1. Everything else (numbers, text, images) is built on top of bits by agreeing on a convention for what patterns of bits mean. For text, the convention has two layers:

1. **Unicode** gives every character in every writing system a number, called its **code point**. `A` is 65, `é` is 233, `€` is 8364, 😊 is 128512. There are over a million possible code points.
2. An **encoding** says how to write those numbers as bytes. A **byte** is a group of 8 bits, which can hold a number from 0 to 255. Code points can be much bigger than 255, so we need rules for spreading one code point over several bytes. **UTF-8** is the encoding used almost everywhere: files, web pages, network messages, and the tokenizer you'll debug on Day 9.

Turning text into bytes is called **encoding**, and turning bytes back into text is **decoding**. Python does both for you with `.encode()` and `.decode()`, but the exam brief explicitly mentions UTF-8, so you may be handed a hand-written encoder or decoder with a wrong number in it. To find that number, you need to understand the bit-level rules. That's what today is for. It's the most technical day of the course, so take it slowly, and do the hand calculations on paper.

1 Bits, binary and hexadecimal: a gentle start

1.1 Counting in binary

We normally write numbers in **base 10**: each digit position is worth ten times the one to its right (ones, tens, hundreds…). **Binary** is base 2: each position is worth *twice* the one to its right, and the only digits are 0 and 1. The positions in one byte (8 bits) are worth, from left to right:

```
position value:  128  64  32  16   8   4   2   1
bits:              1   1   0   0   0   0   1   1
```

To read a binary number, add up the values of the positions that hold a 1. Here: 128 + 64 + 2 + 1 = **195**. The leftmost bit, worth the most, is called the **most significant bit**. The rightmost one is the **least significant bit**. In Python, you write a binary number with the prefix `0b`, and `bin()` shows a number in binary:

```
>>> 0b11000011
195
>>> bin(195)
'0b11000011'
>>> f"{5:08b}"         # 5 as exactly 8 binary digits, padded with zeros
'00000101'
```

1.2 Hexadecimal: a shorthand for binary

Long strings of 0s and 1s are hard to read, so programmers usually write bytes in **hexadecimal** (base 16). Hex digits go 0–9 and then `a`–`f` for 10–15. The big advantage: **one hex digit is exactly 4 bits**, so one byte is exactly two hex digits, and you can convert by splitting the byte in halves:

```
1100 0011
   C    3      ->  0xC3   (C = 12 = 8+4,  3 = 2+1)
```

Python writes hex with the prefix `0x`:

```
>>> 0xC3
195
>>> hex(195)
'0xc3'
>>> int("c3", 16)      # read a hex string
195
>>> int("11000011", 2) # read a binary string
195
```

It's worth memorising the 16 hex digits as 4-bit patterns, at least these: `8 = 1000`, `C = 1100`, `E = 1110`, `F = 1111`, `3 = 0011`, `7 = 0111`. They're exactly the patterns UTF-8 uses.

2 `str` and `bytes` in Python

Python keeps text and raw data strictly apart, with two different types:

- **`str`** is text: a sequence of characters (code points). That's what you used on Day 2.
- **`bytes`** is raw data: a sequence of numbers, each between 0 and 255.

You move from one to the other explicitly: `.encode("utf-8")` turns a `str` into `bytes`, and `.decode("utf-8")` turns `bytes` back into a `str`.

```
>>> s = "héllo"
>>> b = s.encode("utf-8")
>>> b
b'h\xc3\xa9llo'
>>> b.decode("utf-8")
'héllo'
```

Look at how Python displays the bytes: `b'h\xc3\xa9llo'`. The `b` in front marks it as bytes. Bytes that happen to be ordinary ASCII characters (like `h` and `l`) are shown as those characters, for readability. The others are shown as `\x` followed by two hex digits. So `\xc3` is the byte 0xC3 = 195. The character é became *two* bytes, `C3` and `A9`, and section 4 will show you exactly why.

2.1 The differences that cause bugs

	`str`	`bytes`
contains	Unicode **code points** (integers 0..0x10FFFF)	**integers 0..255**
literal	`"héllo"`	`b"h\xc3\xa9llo"` (only ASCII allowed literally)
`x[0]`	`'h'` (a str of length 1)	`104` (**an int!**)
`x[0:1]`	`'h'`	`b'h'` (slicing gives bytes)

	str	bytes
convert	`s.encode("utf-8")` → bytes	`b.decode("utf-8")` → str
mutable version	—	`bytearray`

The two rows that cause the most trouble: **Length.** `len` of a string counts *characters*. `len` of bytes counts *bytes*. As soon as there's a non-ASCII character, they differ:

```
>>> len("héllo")
5
>>> len("héllo".encode("utf-8"))
6
```

So when a spec describes a binary format with "a length prefix giving the size in bytes", the length must be `len(text.encode("utf-8"))`, not `len(text)`. Mixing them up works perfectly for English text and breaks on é, which is why tests always include a non-ASCII example. **Indexing.** Taking one position of a `bytes` object gives you an **int**, not a one-byte `bytes`. Slicing gives you `bytes`:

```
>>> b = b"hello"
>>> b[0]
104
>>> b[0:1]
b'h'
>>> b[0] == b"h"        # an int is never equal to a bytes object
False
>>> b[0] == ord("h")    # compare with the number instead
True
```

Code that writes `if data[i] == b"a":` is always false, a classic bug in parsers.

2.2 Making and inspecting bytes

```
>>> list(b"h\xc3\xa9")        # see the numbers inside
[104, 195, 169]
>>> bytes([104, 105])         # build bytes from a list of numbers
b'hi'
>>> b"ab" + bytes([0xC3, 0xA9])
b'ab\xc3\xa9'
>>> b"abc".hex()              # readable hex text
'616263'
>>> bytes.fromhex("c3 a9")    # and back
b'\xc3\xa9'
```

`bytes` can't be changed once made (like `str`). When you build output one byte at a time, use a **`bytearray`**, the changeable version, and convert at the end:

```
>>> out = bytearray()
>>> out.append(0xC3)          # add one byte (a number 0..255)
>>> out.extend(b"\xa9")       # add several bytes
>>> bytes(out)
b'\xc3\xa9'
```

And note that a string and its bytes are never equal, even when they "say" the same thing:

```
>>> "é" == "é".encode()
False
```

2.3 When decoding goes wrong

Not every sequence of bytes is valid UTF-8. The byte `0xFF`, for example, can never appear in UTF-8. Decoding invalid data raises a `UnicodeDecodeError` by default. The `errors=` argument changes that:

```
>>> b"\xff".decode("utf-8")
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xff in position 0: invalid start
    byte
>>> b"\xffab".decode("utf-8", errors="replace")
'\ufffdab'
>>> b"\xffab".decode("utf-8", errors="ignore")
'ab'
```

- `errors="strict"` (the default) raises.
- `errors="replace"` puts the replacement character `\ufffd` (U+FFFD) where the bad bytes were, and carries on. Tokenizers use this so a stray bad byte never crashes them.
- `errors="ignore"` silently drops bad bytes.

Decode errors are ValueErrors

`UnicodeDecodeError` ⊂ `UnicodeError` ⊂ `ValueError` — so `assertRaises(ValueError)` catches it.

(The ⊂ sign means "is a kind of". `UnicodeDecodeError` is a subclass of `ValueError`, as explained on Day 4, section 7.)

2.4 Never decode in pieces

Never decode per chunk

Multi-byte chars split across chunks — decoding each token/chunk separately breaks:

```
b = "é".encode()          # b'\xc3\xa9'
b[:1].decode("utf-8", errors="replace") + b[1:].decode("utf-8", errors="replace")   #
    '\ufffd\ufffd'  WRONG
(b[:1] + b[1:]).decode()  # 'é'  — always concatenate bytes FIRST, then decode once
```

This exact bug is classic in tokenizer `decode()` functions. Why does it break? The first byte, `\xc3`, means "a character starts here, and it continues in the next byte". On its own, that's an incomplete character, so it becomes `\ufffd`. The second byte, `\xa9`, means "I'm the continuation of a character", but on its own it has nothing to continue, so it also becomes `\ufffd`. Only *together* are they é. A tokenizer stores each token as some bytes, and a token boundary can fall in the middle of a character. That's why the correct decode joins all the bytes first, then decodes once.

3 Bit operations: working with individual bits

To build or take apart UTF-8 bytes, you need to work on individual bits inside a number. Python has operators for that. Each one is easiest to understand with the two numbers written in binary, one above the other.

3.1 AND (&): keep only certain bits

`a & b` produces a 1 only in the positions where **both** a and b have a 1.

```
  1100 0011     (0xC3)
& 0011 1111     (0x3F)
-----------
  0000 0011     (0x03)
```

The most common use is with a **mask**: a number that has 1s exactly where you want to look. `& mask` keeps those bits and sets all the others to 0. In the example, the mask `0x3F` keeps the low 6 bits and throws away the top 2.

```
>>> 0xC3 & 0x3F
3
```

3.2 OR (|): switch certain bits on

`a | b` produces a 1 where **either** number has a 1. You use it to *set* bits, for example to stamp a fixed pattern onto the top of a byte:

```
  1000 0000     (0x80)
| 0010 1001     (0x29)
-----------
  1010 1001     (0xA9)
```

3.3 XOR (^) and NOT (~)

`a ^ b` produces a 1 where the two numbers *differ*, so it flips the bits that are set in the mask. `~a` flips every bit. Because Python's integers have no fixed size, `~a` comes out as the negative number `-a - 1`, so only use it inside expressions like `x & ~mask`. You'll rarely need either for UTF-8.

3.4 Shifts (<< and >>): slide bits left or right

`a << n` moves every bit n places to the left, filling with zeros on the right. That's the same as multiplying by 2^n. `a >> n` moves every bit n places to the right, and the bits that fall off the end are lost. That's the same as dividing by 2^n and rounding down.

```
>>> bin(0b11 << 6)       # 11 becomes 11000000
'0b11000000'
>>> bin(0b11101001 >> 6) # 11101001 becomes 11 (the low 6 bits fall off)
'0b11'
```

In UTF-8 code, shifts are how you move groups of bits into position. The decoder shifts the code point left by 6 to make room for 6 new bits. The encoder shifts right to bring the top bits down into a byte.

3.5 Common bit recipes

```
0b1010, 0xFF, 0o17          # binary, hex, octal literals
bin(10), hex(255)           # '0b1010', '0xff'
f"{10:08b}", f"{255:02x}"   # '00001010', 'ff'
int("1010", 2), int("ff", 16)

a & b     # AND  — keep bits:      x & 0x3F  keeps the low 6 bits
a | b     # OR   — set bits:       0x80 | 0x2A  → 0b10101010
a ^ b     # XOR  — flip bits
~a        # NOT  (-a-1 in Python, beware)
a << n    # shift left  = a * 2**n
a >> n    # shift right = a // 2**n

(x >> i) & 1            # bit i of x
x | (1 << i)            # set bit i
x & ~(1 << i)           # clear bit i
(x >> start) & ((1 << length) - 1)   # extract `length` bits starting at bit `start`
x & (x - 1) == 0        # power of two test (x > 0)
```

How to read the recipes at the bottom (bits are numbered from 0 at the right):

- **Read bit `i`:** shift it down to position 0, then keep only that bit with `& 1`. The result is 0 or 1.
- **Set bit `i`:** `1 << i` is a number with a single 1 at position `i`. OR-ing it in switches that bit on.
- **Clear bit `i`:** `~(1 << i)` has 1s everywhere *except* position `i`. AND-ing with it switches just that bit off.
- **Extract a field:** shift the field down to the bottom, then mask off `length` bits. `(1 << length) - 1` is a row of `length` ones. For example, `(1 << 6) - 1` is `0b111111` = `0x3F`.

3.6 The masks of UTF-8

These masks are used again and again in UTF-8 code. Learn to read them as bit patterns, not as numbers:

mask	binary	used for
`0x80`	`1000 0000`	high bit: ASCII if `b & 0x80 == 0`
`0xC0`	`1100 0000`	`b & 0xC0 == 0x80` → continuation byte
`0xE0`	`1110 0000`	`b & 0xE0 == 0xC0` → 2-byte lead
`0xF0`	`1111 0000`	`b & 0xF0 == 0xE0` → 3-byte lead
`0xF8`	`1111 1000`	`b & 0xF8 == 0xF0` → 4-byte lead
`0x3F`	`0011 1111`	payload of a continuation byte (6 bits)
`0x1F`	`0001 1111`	payload of a 2-byte lead (5 bits)
`0x0F`	`0000 1111`	payload of a 3-byte lead (4 bits)
`0x07`	`0000 0111`	payload of a 4-byte lead (3 bits)

The first five rows are **tests**: "do the top bits of this byte have a certain pattern?" Take the test `b & 0xE0 == 0xC0`. The mask `1110 0000` keeps only the top 3 bits of `b`, and comparing with `1100 0000` checks that those 3 bits are `110`. Section 4 explains why `110` matters. The last four rows are **extractions**: they keep the low bits that carry the actual data, and throw away the pattern bits at the top.

3.7 Operator precedence: which operation happens first

Operator precedence is the rule that decides the order of operations when there are no parentheses, the same way `2 + 3 * 4` means `2 + (3 * 4)` in school maths. For Python's bit operators: **Operator precedence** (high → low): `+ -` → `<< >>` → `&` → `^` → `|` → comparisons (`== < in ...`) → `not` → `and` → `or`. Consequences:

- `b & 0xC0 == 0x80` is `(b & 0xC0) == 0x80` ✔ (unlike C, where it's a bug).
- `x >> 6 & 0x3F` is `(x >> 6) & 0x3F` ✔.
- `1 << n - 1` is `1 << (n - 1)` — arithmetic binds tighter than shifts. Possibly not what you meant.
- `a + b << 2` is `(a + b) << 2`.
- `cp << 6 | b & 0x3F` is `(cp << 6) | (b & 0x3F)` ✔ — but `cp << 6 + b & 0x3F` is garbage.

The good news is that the UTF-8 tests like `b & 0xC0 == 0x80` work as written in Python. The danger is mixing + or − with shifts. In the last example, `6 + b` is computed first, so the code shifts by a huge amount and produces nonsense. If a decoded value looks absurdly large, put parentheses around every operation in the suspicious line and see whether the meaning changes.

> **Parenthesise bit code**
>
> When in doubt, add parentheses.

4 How UTF-8 works

4.1 The idea

UTF-8 has to solve two problems at once:

- Write any code point (up to about 1.1 million) using bytes that each hold only 0–255.
- Let a program reading the bytes always know **where each character starts and how long it is**, even if it starts reading in the middle.

Its solution is to use 1, 2, 3 or 4 bytes per character, and to reserve the **top bits of every byte as a label**:

- A byte starting with `0` is a complete character on its own. These are the 128 ASCII characters (English letters, digits, punctuation), so plain English text in UTF-8 is exactly one byte per character.
- A byte starting with `110`, `1110` or `11110` is the **lead byte** of a character that is 2, 3 or 4 bytes long. *The number of 1s at the start tells you the total length.*
- A byte starting with `10` is a **continuation byte**: it carries more bits of the character that the lead byte started.

The remaining bits in each byte, after the label, are the **payload**: the actual bits of the code point.

4.2 The table

A code point `cp` is encoded with 1–4 bytes:

Code point range	Bytes	Byte 1	Byte 2	Byte 3	Byte 4	payload bits
U+0000 – U+007F	1	`0xxxxxxx`				7
U+0080 – U+07FF	2	`110xxxxx`	`10xxxxxx`			11
U+0800 – U+FFFF	3	`1110xxxx`	`10xxxxxx`	`10xxxxxx`		16
U+10000 – U+10FFFF	4	`11110xxx`	`10xxxxxx`	`10xxxxxx`	`10xxxxxx`	21

(`U+00E9` is the standard way of writing a code point: `U+` followed by the number in hex. So U+00E9 is 0xE9 = 233, which is é.) The `x`s are where the payload bits go. Count them to see where the ranges come from. A 2-byte character has 5 + 6 = 11 payload bits, enough for numbers up to $2^{11} - 1 = 0x7FF$. A 3-byte character has 4 + 6 + 6 = 16 bits, up to 0xFFFF. A 4-byte character has 3 + 6 + 6 + 6 = 21 bits, which comfortably covers the Unicode maximum of 0x10FFFF.

> **Most-significant bits first**
>
> The bits of `cp` are written **most-significant first**: the lead byte gets the top bits, each continuation byte the next 6 bits.

4.3 Encoding by hand

Here's the recipe for turning a code point into bytes:

1. Look up the range in the table. That tells you how many bytes you need, and how many payload bits.
2. Write the code point in binary, padded on the left with zeros to exactly that many bits.
3. Cut the bits into groups: the lead byte's group first (5, 4 or 3 bits), then groups of 6.
4. Put the label in front of each group: `110`/`1110`/`11110` on the first, `10` on each of the others.

Let's do it three times. Use pen and paper alongside. **'é' = U+00E9** → 0xE9 = `1110 1001`, in range 0x80–0x7FF → 2 bytes, 11 payload bits: `000 1110 1001` → split 5 | 6 → `00011` | `101001` → `110 00011` = 0xC3, `10 101001` = 0xA9 → **C3 A9** ✔ (`"é".encode() == b'\xc3\xa9'`) In words: 0xE9 is 233, which is more than 127 but less than 0x7FF, so it needs 2 bytes and 11 bits. Written as 11 bits, 233 is `00011101001`. The first 5 bits, `00011`, go after `110` to give `11000011` = C3. The last 6, `101001`, go after `10` to give `10101001` = A9. **'€' = U+20AC** → `0010 0000 1010 1100`, 3 bytes, split 4 | 6 | 6 → `0010` | `000010` | `101100` → `1110 0010` E2, `10 000010` 82, `10 101100` AC → **E2 82 AC** **'😀' = U+1F600** → 21 bits `0 0001 1111 0110 0000 0000` → split 3|6|6|6 → `000 011111 011000 000000` → F0 9F 98 80 Check your work in the REPL:

```
>>> "€".encode().hex(" ")
'e2 82 ac'
>>> "😀".encode().hex(" ")
'f0 9f 98 80'
```

Being able to produce these bytes by hand is what lets you judge a failing test: you can work out what the right answer is, and see which group of bits the broken code got wrong.

4.4 Encoding in code

```python
def encode_cp(cp):
    if cp < 0x80:
        return bytes([cp])
    if cp < 0x800:
        return bytes([0xC0 | (cp >> 6),
                      0x80 | (cp & 0x3F)])
    if cp < 0x10000:
        return bytes([0xE0 | (cp >> 12),
                      0x80 | ((cp >> 6) & 0x3F),
                      0x80 | (cp & 0x3F)])
    return bytes([0xF0 | (cp >> 18),
                  0x80 | ((cp >> 12) & 0x3F),
                  0x80 | ((cp >> 6) & 0x3F),
                  0x80 | (cp & 0x3F)])
```

This is exactly the pen-and-paper recipe, written with shifts and masks. Take the 3-byte branch, with '€' = 0x20AC = `0010 000010 101100`:

- **Lead byte:** `cp >> 12` slides the number right by 12, dropping the two 6-bit groups and leaving the top 4 bits, `0010`. Then `0xE0 |` stamps the label `1110` on top: `1110 0010` = E2.
- **Middle byte:** `cp >> 6` drops the last group, leaving `0010 000010`. Then `& 0x3F` keeps only the low 6 bits, `000010`, and `0x80 |` adds the label `10`: `1000 0010` = 82.
- **Last byte:** `cp & 0x3F` keeps the lowest 6 bits, `101100`, and the label gives `1010 1100` = AC.

The `if` boundaries come straight from the table: `< 0x80` fits in 1 byte, `< 0x800` in 2, `< 0x10000` in 3. They use *strictly less than*. Typical planted bugs are `<=` on a boundary, a wrong shift (`>> 8` instead of `>> 6`), or a missing `& 0x3F`, which lets extra high bits leak into a continuation byte.

4.5 Decoding: reading bytes back into characters

Decoding runs the recipe backwards. It also has to **validate**, because the bytes it receives might be broken. Here are the steps for one character, starting at the current position:

1. Look at the lead byte `b0`:
   - `b0 < 0x80` → 1 byte, `cp = b0`
   - `b0 & 0xE0 == 0xC0` → 2 bytes, `cp = b0 & 0x1F`
   - `b0 & 0xF0 == 0xE0` → 3 bytes, `cp = b0 & 0x0F`
   - `b0 & 0xF8 == 0xF0` → 4 bytes, `cp = b0 & 0x07`
   - otherwise (a continuation byte `10xxxxxx` or `0xF8..0xFF`) → **invalid start byte**
2. For each of the following `n-1` bytes: check it exists (else **truncated**) and that `b & 0xC0 == 0x80` (else **invalid continuation**); then `cp = (cp << 6) | (b & 0x3F)`.
3. Validate the result:
   - **overlong**: the cp would fit in fewer bytes (e.g. `C0 AF` decodes to `/` with 2 bytes — forbidden). Minimums: 2 bytes ≥ 0x80, 3 bytes ≥ 0x800, 4 bytes ≥ 0x10000. (So `C0` and `C1` can never appear.)
   - **surrogates** U+D800–U+DFFF are forbidden in UTF-8.
   - **> U+10FFFF** is forbidden (so `F5..FF` can never appear).

The heart of step 2 is the line `cp = (cp << 6) | (b & 0x3F)`. `cp << 6` slides the bits collected so far 6 places to the left, making room at the bottom. `b & 0x3F` takes the 6 payload bits of the new byte. `|` drops them into the space. After the last continuation byte, all the payload bits are in place. Each check in step 3 has a reason:

- **Truncated:** the lead byte promised, say, 3 bytes, but the data ends after 2, as when a file was cut off mid-character.

- **Invalid continuation:** a byte that should continue the character doesn't start with `10`, so the character is broken.
- **Overlong:** you *could* encode a small number with more bytes than necessary, by putting zeros in front of it. For example, `/` (0x2F) could be squeezed into the 2-byte pattern as `C0 AF`. UTF-8 forbids this so every character has exactly *one* valid encoding. (Otherwise, a security filter looking for the byte `/` could be fooled by `C0 AF`.) The check is simple: a 2-byte character must be at least 0x80, a 3-byte one at least 0x800, and a 4-byte one at least 0x10000.
- **Surrogates:** the code points U+D800–U+DFFF are reserved for a different encoding (UTF-16) and are not real characters.
- **Too big:** Unicode stops at 0x10FFFF, but 21 bits could hold numbers up to 0x1FFFFF, so the decoder must reject anything above the limit.

4.6 Decoding in code

```
def decode(data):
    out, i = [], 0
    while i < len(data):
        b0 = data[i]
        if b0 < 0x80:       n, cp, lo = 1, b0, 0
        elif b0 & 0xE0 == 0xC0: n, cp, lo = 2, b0 & 0x1F, 0x80
        elif b0 & 0xF0 == 0xE0: n, cp, lo = 3, b0 & 0x0F, 0x800
        elif b0 & 0xF8 == 0xF0: n, cp, lo = 4, b0 & 0x07, 0x10000
        else: raise ValueError(f"invalid start byte at {i}")
        if i + n > len(data): raise ValueError("truncated")
        for k in range(1, n):
            b = data[i + k]
            if b & 0xC0 != 0x80: raise ValueError("invalid continuation")
            cp = (cp << 6) | (b & 0x3F)
        if cp < lo or 0xD800 <= cp <= 0xDFFF or cp > 0x10FFFF: raise ValueError("invalid")
        out.append(chr(cp))
        i += n
    return "".join(out)
```

The variables: `i` is the position of the current lead byte, `n` is the length of this character, `cp` is the code point being assembled, and `lo` is the smallest code point allowed for this length (the overlong check). Let's trace `decode(b"\xc3\xa9")`:

1. `i = 0`, `b0 = 0xC3 = 1100 0011`.
2. Is it `< 0x80`? No. Is `b0 & 0xE0` (= `1100 0000`) equal to `0xC0`? Yes, so it's a 2-byte character: `n = 2`, `cp = 0xC3 & 0x1F = 00011`, `lo = 0x80`.
3. Truncation check: `i + n = 2`, and the data has 2 bytes, so that's fine.
4. `k = 1`: `b = 0xA9 = 1010 1001`. `b & 0xC0 = 1000 0000 = 0x80`, so it's a valid continuation. Then `cp = (00011 << 6) | 101001 = 00011 101001 = 0xE9`.
5. Final checks: `0xE9 >= 0x80` (not overlong), not a surrogate, not too big.
6. `chr(0xE9)` is `'é'`. Then `i = 2` and the loop ends. Result: `'é'`.

This kind of loop, where `i` is a position that moves forward by a different amount each time, is called **cursor-based parsing**. You'll see it again on Day 8 and in the Day 9 tokenizer. When a decoder is broken, usually only *certain* characters break: a wrong mask affects only 3-byte characters, a wrong `lo` only overlong cases. That's why the tests use one example from each length class, and why the failing test's input tells you which branch to look at.

> **Check lead bytes in the right order**
>
> If you test `b0 >= 0xC0` before `b0 >= 0xE0`, every 3- and 4-byte lead is misclassified as 2-byte. Using masks (`& 0xE0 == 0xC0`) avoids this; using `>=` requires checking the largest first.

To see why: a 3-byte lead like `0xE2` is also `>= 0xC0`. If the `>= 0xC0` test comes first, it matches, and the `>= 0xE0` branch is never reached.

4.7 Handy properties that follow from the design

- ASCII is valid UTF-8 unchanged; any byte < 0x80 in UTF-8 is **always** an ASCII char.
- Number of code points in valid UTF-8 = number of bytes that are **not** continuation bytes (`sum(1 for b in data if b & 0xC0 != 0x80)`).
- You can find a character boundary from any position by moving back while `b & 0xC0 == 0x80`. That's how you truncate to N bytes without cutting a character in half.
- Sorting UTF-8 bytes gives the same order as sorting code points.

Why these hold:

- The first follows from the labels: a multi-byte character's bytes all start with `1`, so a byte starting with `0` can only be a standalone ASCII character.
- The second holds because every character has exactly one lead byte, so counting the bytes that are *not* continuation bytes counts the characters.
- The third: continuation bytes are recognisable on their own (they start with `10`), so from any position you can step backwards over them until you reach the lead byte of the current character. To cut some bytes down to at most N bytes without splitting a character, cut at N, then move the cut left while the byte at the cut is a continuation byte.

5 Other Unicode facts (good to know, rarely tested)

You don't need to study these today. Just be aware of them.

- `len("é")` can be 1 (U+00E9) or 2 ("é" = e + combining accent). `unicodedata.normalize("NFC", s)` merges.
- Emoji with skin tones / families are several code points: `len("👍") == 2`.
- `"ß".upper() == "SS"` — case mapping can change length.
- UTF-8 BOM: `b"\xef\xbb\xbf"`; `"utf-8-sig"` strips it.
- `int.to_bytes(4, "big")`, `int.from_bytes(b, "big")` for fixed-width integers (used in binary formats).

```
(258).to_bytes(2, "big")      # b'\x01\x02'
int.from_bytes(b"\x01\x02", "little")  # 513
```

The last item *does* matter, for binary file formats like the one on Day 10. `n.to_bytes(k, order)` writes an integer as exactly `k` bytes, and `int.from_bytes(data, order)` reads it back. The `order` says which byte comes first:

- **big-endian** (`"big"`) puts the most significant byte first, the way we write numbers on paper. 258 = 0x0102 becomes `01 02`.
- **little-endian** (`"little"`) puts it last: `02 01`.

If a file is written with one order and read with the other, you get a wrong number and no error at all: `01 02` read as little-endian is 0x0201 = 513 instead of 258. That makes it a favourite planted bug.

6 The toolbox: techniques the exercises rely on

6.1 An exception that carries extra information

Day 4 (section 7) showed the simplest custom exception, `class MyError(Exception): pass`. The strict decoder in ex03 needs more: its tests check not only *that* it fails, but also *why* and *where*. So the exception object stores those two facts as attributes:

```
class UTF8DecodeError(ValueError):
    def __init__(self, reason, position):
        super().__init__(f"{reason} at position {position}")
        self.reason = reason
        self.position = position
```

Line by line:

- `(ValueError)`: it's a kind of `ValueError` (inheritance, Day 4 section 6), so code that catches `ValueError` also catches it.
- `__init__(self, reason, position)`: it's created with two values, as in `raise UTF8DecodeError("surrogate", 4)`.
- `super().__init__(message)`: lets the original `ValueError.__init__` store the readable message, the text shown in a traceback. Skipping this line gives an exception with an empty message.
- `self.reason = ...`, `self.position = ...`: store the details as ordinary attributes, so anyone holding the exception object can read them.

How does a test get hold of the exception object? `assertRaises` used with `as` (Day 6 covers it in detail):

```
with self.assertRaises(UTF8DecodeError) as ctx:
    utf8_decode(b"\xc3")
self.assertEqual(ctx.exception.reason, "truncated sequence")
self.assertEqual(ctx.exception.position, 0)
```

`ctx.exception` is the exception that was raised, so its `.reason` and `.position` can be checked.

6.2 Setting several variables per branch

The decoder has to remember three facts that depend on the lead byte: how long the sequence is, the payload bits from the lead byte, and the smallest code point that length may encode. Each branch sets all three in one line using unpacking (Day 1, section 8):

```
if b0 & 0xE0 == 0xC0:
    length, cp, minimum = 2, b0 & 0x1F, 0x80
elif b0 & 0xF0 == 0xE0:
    length, cp, minimum = 3, b0 & 0x0F, 0x800
```

`b0 & 0xE0 == 0xC0` works as you'd hope: `&` binds more tightly than `==` (see the precedence table in section 3), so it means `(b0 & 0xE0) == 0xC0`, "do the top three bits read 110?".

6.3 Looking ahead from a position: `data[i + k]`

A multi-byte character occupies `data[i]`, `data[i + 1]`, ..., `data[i + length - 1]`. The continuation bytes are `data[i + k]` for `k` in `range(1, length)`, which skips `k = 0`, the lead byte. Before looking ahead, check that the bytes exist: `if i + length > len(data)`, otherwise `data[i + k]` raises `IndexError` at the end of the data. After a character is done, jump over all of it with `i += length`, not `i += 1`.

6.4 Building bytes: `bytes([...])` and `bytearray +=`

`bytes([n1, n2, ...])` turns a list of numbers 0–255 into a bytes object (section 2). Each element of the list can be a whole expression, so a 2-byte UTF-8 character is built in one call:

```
bytes([0xC0 | (cp >> 6), 0x80 | (cp & 0x3F)])
```

To join many such pieces, add them to a `bytearray` with `+=`, which appends *all* the bytes of the right-hand side (like `extend`), and convert to `bytes` once at the end:

```
out = bytearray()
for ch in s:
    out += encode_code_point(ord(ch))
return bytes(out)
```

6.5 Counting bytes and characters with a generator

Section 4's "handy properties" say every character starts with exactly one byte that is *not* a continuation byte (`10xxxxxx`). So "how many characters are in these bytes?" is the counting pattern from Day 1:

```
sum(1 for b in data if b & 0xC0 != 0x80)
```

Looping over a `bytes` object gives **numbers** (section 2), which is why `b & 0xC0` works directly. Likewise, "the positions where a character starts" is a comprehension with `enumerate`:

```
[i for i, b in enumerate(data) if b & 0xC0 != 0x80]
```

6.6 Stop as soon as the limit is passed

"The longest prefix of `s` that fits in `max_bytes` bytes" is a running total (Day 1) that stops early. Walk through the characters with `enumerate`, add each character's byte length, and the moment the total goes over the limit, the answer is everything *before* the current character, `s[:i]`. If the loop finishes without going over, the whole string fits.

6.7 Taking a number apart bit by bit, and building one

Two loops appear in the bit exercises. The first **reads** the bits of `n` from the bottom, one per round: `n & 1` is the lowest bit, and `n >>= 1` (short for `n = n >> 1`) throws it away so the next bit moves to the bottom. The loop `while n:` ends when no 1 bits are left. This is the binary version of Day 1's `divmod(n, 10)` digit loop. The second **builds** a number bit by bit: `out = (out << 1) | bit` shifts everything built so far one place left, making room at the bottom, and puts the new bit there. Feeding in the bits 1, 1, 0 gives `0b1`, then `0b11`, then `0b110`. Reading bits from the bottom of one number and pushing them onto the bottom of another reverses their order. To print a number as a fixed number of binary digits without `format`, go through the bit positions from the top (`range(width - 1, -1, -1)`, Day 1 section 4) and turn each bit into `"0"` or `"1"` with `str(...)`. A neat test: `n & (n - 1)` clears the lowest 1 bit of `n` (subtracting 1 turns `...1000` into `...0111`, and AND-ing the two wipes out that part). A power of two has exactly one 1 bit, so for it the result is 0.

6.8 A flag and `break`

The bonus `decode_replace` checks continuation bytes in a loop, and stops at the first bad one. A boolean "flag" variable records whether the loop finished normally:

```
ok = True
for k in range(need):
    if <this byte is bad>:
        ok = False
        break
    ...
# here, ok tells you whether every byte was fine
```

(Day 1's `for ... else` could do the same, but the flag is easier to read.)

Exercises

Practice lab

Work in `exercises/`. Read each docstring as the spec. Run `python run.py 5` after every change; do `ex05` with a 20-minute timer.

File	Type	What
`exercises/ex01_bits.py`	implement	bit helpers (get/set/clear, extract, popcount, padded binary)
`exercises/ex02_utf8_encode.py`	implement	encode code points and strings to UTF-8 **without** `.encode()`
`exercises/ex03_utf8_decode.py`	implement	strict decoder with all error checks **without** `.decode()`; bonus: `errors="replace"` semantics
`exercises/ex04_utf8_tools.py`	implement	truncate to N bytes, char boundaries, byte↔char offsets, hex dump
`exercises/ex05_fix_me.py`	**debug**	a UTF-8 decoder with 6 planted bugs (exam-style!)

The order is deliberate:

- **ex01** makes the bit operators automatic. "Popcount" means counting how many bits of a number are 1.
- **ex02** and **ex03** have you write the encoder and the decoder yourself, *without* using Python's built-in `.encode()`/`.decode()`. Writing them is the best way to understand every line, and the tests compare your results with Python's. Try to write them **without scrolling back** to the code in section 4: work from the table and the step-by-step recipes, and look at the code only if you're stuck.
- **ex04** applies the properties from the end of section 4. "Byte ↔ char offsets" means converting a position counted in bytes into a position counted in characters, and back.
- **ex05** is an exam simulation: someone else's decoder, broken in six places.

Which tools each exercise needs (section numbers in brackets):

Exercise	Tools
ex01 bits	the recipes and masks (3), reading/building numbers bit by bit (6), `f"{n:08b}"` (1)

Exercise	Tools
ex02 encoder	the table and "encoding by hand" (4), `bytes([...])` and `bytearray +=` (6), `ord` (Day 2)
ex03 decoder	lead-byte checks and the error order (4), the exception with `reason`/`position` (6), `data[i + k]` look-ahead (6), a flag and `break` for the bonus (6)
ex04 tools	`.encode("utf-8")` (2), `enumerate` and stopping early (6), counting with `sum(1 for ...)` (6), `f"{b:02x}"` and `" ".join`
ex05 fix-me	all of section 4

```
python run.py 5
```

Do **ex05 with a timer**: target 20 minutes. Use the worked examples above to compute expected values by hand, and print intermediate `cp` values in binary (`f"{cp:021b}"`). Printing `cp` in binary after every step of the loop shows exactly where bits go wrong. A wrong mask shows up as extra or missing bits at the top, and a wrong shift shows up as the right bits in the wrong place. The bonus `decode_replace` in ex03 is hard (it must match Python's "maximal subpart" rule exactly). Skip it if you're short on time — it's checked against Python's own decoder on random bytes. (The "maximal subpart" rule decides how many `⊠` characters a broken sequence turns into. Roughly: the longest prefix that *could* have started a valid character becomes a single `⊠`, and decoding resumes right after it. So `E2 82 41`, a 3-byte character cut short and followed by `A`, becomes `⊠A`: the `E2 82` part is one `⊠`.)

End-of-day checklist

Tick every box

- [ ] I can encode U+00E9, U+20AC, U+1F600 by hand.
- [ ] I know the 4 lead-byte patterns, the continuation pattern, and the payload masks.
- [ ] I know the 4 error classes: invalid start, invalid/missing continuation, overlong, surrogate/out-of-range.
- [ ] I know `bytes[i]` is an int and to decode the concatenated bytes, not per-chunk.

Exam lens

- Decode the lead byte, validate each continuation, then reject overlong, surrogate, and out-of-range values.
- Never decode per chunk; concatenate the bytes first and decode once.
- Hand-compute `C3 A9`, `E2 82 AC`, and `F0 9F 98 80` until the masks are automatic.

Tonight's bug journal

Where	Symptom	Root cause / fix

One row per bug. Re-read this page on Day 10.